

Data-Driven Lateral Vehicle Control for Autonomous Driving: A Comparative Study of Koopman Operator MPC, AR-Model MPC, and Feedforward-PID Controllers on the Comma AI Controls Challenge

Jaydeep Khekar
Dept. of Mechanical Engineering
Clemson University
Clemson, SC, USA
jkhekar@clemson.edu

Shravan Prabhakar Vidyakshmi
Dept. of Mechanical Engineering
Clemson University
Clemson, SC, USA
shravap@clemson.edu

Abstract—Autonomous lateral vehicle control requires controllers that generalize across diverse road conditions, vehicle dynamics, and speed profiles while simultaneously minimizing tracking error and passenger discomfort. This paper presents a comprehensive comparative study of three data-driven lateral acceleration controllers evaluated on the Comma AI Controls Challenge within the TINYPHYSICS black-box physics simulator, an autoregressive neural network trained on real openpilot driving data.

The three controllers are: (1) a Feedforward-PID with 10-step future preview and empirically calibrated feedforward gain; (2) an AR-Model Incremental MPC using a closed-form QP in error space with precomputed gain vectors; and (3) a Koopman Operator MPC with a 90-dimensional EDMD-identified plant model, dual-head blended prediction, and random-shooting optimization.

Evaluated over 100 driving segments using the challenge cost $J_{\text{total}} = 50 J_{\text{latAccel}} + J_{\text{jerk}}$: FF-PID achieves the best total cost of 61.62 ($J_{\text{lat}} = 0.848$, $J_{\text{jerk}} = 19.21$); AR-MPC achieves 70.47 ($J_{\text{lat}} = 0.981$, $J_{\text{jerk}} = 21.40$); Koopman-MPC achieves 132.8 ($J_{\text{lat}} = 2.358$, $J_{\text{jerk}} = 14.92$). On a single-segment basis (segment 00000), AR-MPC is best at 67.83, FF-PID second at 85.63, and Koopman-MPC third at 138.7.

The central finding is that the $50 \times \text{latAccel}$ weighting makes preview-based feedforward control extremely competitive when future targets are exactly known. The Koopman operator’s most practical contribution is providing a data-driven 90-dimensional plant model with validated $R^2 = 0.87$ at $H = 15$ open-loop rollout. We document four critical implementation lessons and provide cost distribution histograms and tracking plots for all three controllers.

Index Terms—model predictive control, Koopman operator, EDMD, lateral vehicle control, autonomous driving, AR model, feedforward control, data-driven control, TinyPhysics, comma AI

I. INTRODUCTION

A. Motivation

Lateral vehicle control which is for steering a vehicle to follow a desired lateral acceleration trajectory is a foundational

challenge in autonomous driving. Real-world lateral dynamics are nonlinear: the steer-to-lateral-acceleration gain changes with vehicle speed through tire cornering stiffness; the system exhibits significant autoregressive inertia; and road roll introduces persistent disturbances. Classical PID controllers are reactive and fixed-gain, struggling to balance tracking accuracy against ride comfort.

Model Predictive Control (MPC) addresses these limitations through predictive optimization over a finite horizon, but requires an accurate plant model. When vehicle dynamics are represented by a black-box neural simulator as in the Comma AI Controls Challenge, the classical plant identification fails, motivating data-driven approaches such as Koopman operator lifting and autoregressive system identification.

B. The Comma AI Controls Challenge

The Comma AI Controls Challenge [1] evaluates lateral controllers within TINYPHYSICS: an autoregressive transformer model trained on real openpilot driving data that serves as the vehicle physics engine. At each 100 ms timestep, the controller receives the current state and a 5-second preview of future target lateral accelerations, returning a normalized steer command $u \in [-2, 2]$.

The composite evaluation metric is:

$$J_{\text{total}} = 50 J_{\text{latAccel}} + J_{\text{jerk}} \quad (1)$$

with:

$$J_{\text{latAccel}} = \frac{100}{N} \sum_{t=100}^{500} (\hat{a}_{\text{lat}}(t) - a_{\text{lat}}(t))^2 \quad (2)$$

$$J_{\text{jerk}} = \frac{100}{N} \sum_{t=101}^{500} \left(\frac{a_{\text{lat}}(t) - a_{\text{lat}}(t-1)}{0.1} \right)^2 \quad (3)$$

The $50\times$ multiplier makes tracking accuracy approximately $10\times$ more impactful than jerk per unit error. It is a fact that fundamentally shapes optimal controller design.

C. Contributions

- 1) Complete technical description of three data-driven lateral controllers at different complexity levels.
- 2) Rigorous empirical evaluation on 100 segments with cost distribution histograms and debug tracking plots for all three controllers.
- 3) Full Koopman training pipeline analysis: 90-dimensional EDMD, delay embedding, dual-head readout, and spectral stabilization.
- 4) Four documented implementation lessons relevant to any practitioner applying MPC to black-box neural simulators.
- 5) Ablation analysis quantifying the individual contribution of each architectural component.

D. Paper Organization

Section II describes the simulation environment. Section III reviews related work. Section IV presents all three controllers. Section V details the Koopman training pipeline. Section VI describes the experimental setup. Section VII presents and analyzes results with figures. Section VIII documents implementation lessons. Section IX discusses design trade-offs. Section X concludes with future work.

II. SIMULATION ENVIRONMENT

A. TinyPhysics Simulator

TINYPHYSICS is an ONNX autoregressive transformer that predicts the next lateral acceleration from the last 20 context steps:

$$a_{\text{lat}}(t+1) = f_{\theta}(\{s_{\tau}, u_{\tau}, a_{\tau}\}_{\tau=t-19}^t) \quad (4)$$

Output is decoded through a 1024-bin tokenizer over $[-5, 5]$ m/s² with temperature $T = 0.8$, and clipped to prevent jumps:

$$\hat{a}(t) = \text{clip}(\hat{a}(t), a(t-1) - 0.5, a(t-1) + 0.5) \quad (5)$$

The ± 0.5 m/s²/step constraint is a key property: the simulator itself imposes jerk limiting, so aggressive steer commands are partially smoothed before they affect the cost.

B. State and Action Space

TABLE I: Controller Input Signals at Each Timestep

Signal	Description	Units
roll_lataccel	Lat. accel from road roll	m/s ²
v_ego	Longitudinal vehicle speed	m/s
a_ego	Longitudinal acceleration	m/s ²
target_lataccel	Reference lateral acceleration	m/s ²
steer_command	Steer input (right-positive)	$[-2, 2]$
future_plan	Next 50 steps of all signals	—

Critical sign convention. The simulator negates `steerCommand` when loading CSV data:

```
steer_command = -df['steerCommand'].values
```

This converts from openpilot’s left-positive logging convention to the simulator’s right-positive control convention. Any model trained on raw CSV steer values must account for this sign inversion.

C. Dataset Statistics

TABLE II: Dataset Statistics — 50 Segments, 29,978 Rows

Signal	Mean	Std	Min	Max
v_{ego} (m/s)	22.62	9.86	−0.07	36.79
a_{ego} (m/s ²)	0.002	0.45	−2.95	2.16
ϕ_{roll} (rad)	0.012	0.028	−0.077	0.084
$\hat{a}_{\text{lat}}$ (m/s ²)	−0.002	0.41	−2.76	3.78
<code>steerCommand</code>	0.055	0.23	−0.88	0.82

The dataset is dominated by highway driving ($\bar{v} = 22.6$ m/s) with small lateral accelerations ($\sigma = 0.41$ m/s²) and minimal road roll ($\sigma = 0.028$ rad). Steer command data is available for only ≈ 100 of 600 rows per segment.

D. Simulation Loop

Algorithm 1 TINYPHYSICS Control Loop

Init: Load CSV, seed RNG, replay steps 0–19 (context).
for $t = 20, \dots, 600$ **do**
 Read $v_{\text{ego}}, a_{\text{ego}}, \phi$ and $\hat{a}(t)$.
 Assemble `FuturePlan` (next 50 steps).
 $u_t \leftarrow \text{controller.update}(\hat{a}(t), a(t), \text{state}, \text{fp})$
 $u_t \leftarrow \text{clip}(u_t, -2, 2)$
 if $t < 100$ **then**
 Override: $u_t \leftarrow \text{recorded steer (warmup)}$
 end if
 TinyPhysics ONNX $\rightarrow$ decode $\rightarrow$ clip $\rightarrow a(t+1)$
end for
Compute costs over steps 100–500.

III. RELATED WORK

LTV-MPC for lateral control. Falcone et al. [2] show that successive local linearization (LTV-MPC) reduces computational burden while preserving accuracy on icy roads at 21 m/s, directly motivating the AR(1) error-space MPC in this work.

Velocity-scheduled adaptive MPC. Kebbati et al. [3] show that fixed-parameter MPC degrades under speed variation, motivating the velocity-dependent weighting used in this work.

Learning-based adaptive MPC. The companion work [4] demonstrates 78% MSE reduction using neural network-adapted MPC weights, motivating Koopman lifting as a data-driven plant model.

Koopman operator methods. Williams et al. [7] formalize EDMD for Koopman approximation, providing the theoretical backbone for Section V. Brunton and Kutz [8] survey Koopman-based control applications.

Hybrid RL-MPC. Wu et al. [5] use MPC as expert demonstrator for DDPG training, motivating future AR-MPC-based RL warm-start.

Vehicle dynamics reference. Rajamani [6] derives the two-DOF bicycle model, informing AR(1) parameterization in this work.

IV. CONTROLLER ARCHITECTURES

A. Controller 1: Feedforward-PID with Future Preview

1) *Design Philosophy:* The FF-PID separates steady-state tracking (feedforward) from disturbance rejection (PID). Since future targets are exactly known via `future_plan`, an aggressive preview weight effectively pre-solves the tracking problem without any optimization.

2) *Preview-Blended Effective Target:*

$$\hat{a}_{\text{eff}}(t) = (1 - \beta) \hat{a}_{\text{lat}}(t) + \beta \frac{1}{N} \sum_{k=1}^N \hat{a}_{\text{lat}}(t + k) \quad (6)$$

with $\beta = 0.949$ and $N = 10$ (1-second preview window). The high future weight means the controller steers toward the 1-second-ahead average trajectory before turns arrive.

3) *Feedforward Calibration:*

$$u_{\text{FF}}(t) = K_{\text{FF}} \hat{a}_{\text{eff}}(t), \quad K_{\text{FF}} = 0.31498 \quad (7)$$

Identified empirically from steady-state openpilot tracking data as the mean steer-to-latAccel ratio during periods of accurate tracking.

4) *PID Residual Correction:*

$$e_t = \hat{a}_{\text{lat}}(t) - a_{\text{lat}}(t) \quad (8)$$

$$I_t = \text{clip}(I_{t-1} + e_t, -5, 5) \quad (9)$$

$$u_{\text{PID}}(t) = 0.970 e_t + 0.076 I_t + 0.5 (e_t - e_{t-1}) \quad (10)$$

5) *Output Smoothing and Rate Limiting:*

$$\text{raw}(t) = u_{\text{FF}}(t) + u_{\text{PID}}(t) \quad (11)$$

$$\text{out}(t) = 0.85 u(t-1) + 0.15 \text{raw}(t) \quad (12)$$

$$u(t) = \text{clip}(\text{out}(t), u(t-1) \pm 0.2, -2, 2) \quad (13)$$

TABLE III: FF-PID Controller Parameters

Parameter	Description	Value
K_{FF}	Feedforward gain	0.31498
β	Future preview weight	0.949
N	Preview steps	10
K_p	Proportional gain	0.970
K_i	Integral gain	0.076
K_d	Derivative gain	0.5
EMA	Output smoothing factor	0.85
Δu_{max}	Per-step rate limit	0.20

B. Controller 2: AR-Model Incremental MPC

1) *Design Philosophy:* The AR-MPC separates steady-state tracking (feedforward preview) from disturbance rejection (incremental MPC). The key insight: including future target changes in the MPC cost causes Δu spikes that exceed rate limits at every step, rendering the optimizer useless (see Lesson 4 in Section VIII). The solution: let the feedforward handle targets; use MPC only for roll disturbance and bias correction.

2) *AR(1) System Identification:* The lateral acceleration dynamics are modeled as:

$$a_{\text{lat}}[t+1] = \alpha a_{\text{lat}}[t] + b u[t] + c_\phi \phi[t] + d \quad (14)$$

with parameters identified from 50 closed-loop PID rollouts: $\alpha = 0.9577$, $b = 0.0605$, $c_\phi = 0.0404$, $d = -0.00783$. The large $\alpha \approx 0.958$ reflects the slow-changing nature of lateral acceleration. The short-horizon gain $b = 0.0605$ implies a steady-state gain of $b/(1 - \alpha) \approx 1.42$, consistent with $1/K_{\text{FF}} \approx 3.17$.

3) *Error-Space Reformulation:* Define $e[t] = a_{\text{lat}}[t] - \hat{a}[t]$ and $\Delta u[t] = u[t] - u_{\text{FF}}[t]$:

$$e[t+1] \approx \alpha e[t] + b_c \Delta u[t] + f[t] \quad (15)$$

where the corrected gain $b_c = (1 - \alpha)/K_{\text{FF}} \approx 0.141$ ensures zero steady-state error, and the disturbance:

$$f[t] = c_\phi \phi[t] + d = 0.0404 \phi[t] - 0.00783 \quad (16)$$

contains only roll and constant bias which are always tiny, never triggering rate-limit saturation.

4) *MPC Formulation Over Horizon $H = 20$:*

$$\mathbf{E} = \mathbf{S}_x e_0 + \mathbf{S}_u \Delta \mathbf{U} + \mathbf{S}_d \mathbf{f} \quad (17)$$

$$J(\Delta \mathbf{U}) = Q \|\mathbf{E}\|^2 + R_\Delta \|\mathbf{D} \Delta \mathbf{U} - \Delta u_{t-1} \mathbf{e}_1\|^2 + R_e \|\Delta \mathbf{U}\|^2 \quad (18)$$

with $Q = 50$, $R_\Delta = 7$, $R_e = 0.02$.

5) *Precomputed Closed-Form Solution:* The QP has a unique analytical solution. All matrices are inverted once at class initialization:

$$\mathbf{M} = \mathbf{S}_u^\top Q \mathbf{S}_u + R_\Delta (\mathbf{D}^\top \mathbf{D}) + R_e \mathbf{I} \quad (19)$$

$$\Delta u^*[0] = k_{e_0} e_0 + \mathbf{k}_f^\top \mathbf{f} + k_{\Delta u} \Delta u_{t-1} \quad (20)$$

Per-step cost is $O(H)$ is a single dot product regardless of system dimension.

6) *Output Composition:*

$$\delta_u = \Delta u^*[0] - 0.020 \int e dt + (-0.05)(e_t - e_{t-1}) \quad (21)$$

$$u(t) = \text{clip}(0.87 u(t-1) + 0.13(u_{\text{FF}} + \delta_u), u(t-1) \pm 0.07, -2, 2) \quad (22)$$

TABLE IV: AR-MPC Controller Parameters

Parameter	Description	Value
α	AR autoregressive coefficient	0.9577
b_c	Corrected gain (error-space)	0.1410
c_ϕ / d	Roll gain / bias	0.0404 / -0.00783
H	MPC horizon	20
$Q / R_\Delta / R_e$	Cost weights	50 / 7 / 0.02
K_{FF} / β	FF gain / preview weight	0.30 / 0.70
EMA	Output smoothing	0.87
Δu_{max}	Per-step rate limit	0.07

C. Controller 3: Koopman Operator MPC

1) *Koopman Operator Theory*: For a nonlinear system $x_{t+1} = f(x_t)$, the Koopman operator $\mathcal{K}$ acts on observable functions $g : \mathcal{X} \rightarrow \mathbb{R}$:

$$\mathcal{K} g(x_t) = g(f(x_t)) = g(x_{t+1}) \quad (23)$$

While f is nonlinear, $\mathcal{K}$ is linear that enables linear control design for nonlinear systems. We approximate it with a finite dictionary $\psi : \mathbb{R}^4 \rightarrow \mathbb{R}^{90}$:

$$z_{t+1} \approx \mathbf{A} z_t + \mathbf{B} u_t, \quad z_t = \psi(x_t) \in \mathbb{R}^{90} \quad (24)$$

2) *Three-Stage Lifting Pipeline*: The raw state $x_t = [v_{\text{ego}}, a_{\text{ego}}, \phi_{\text{roll}}, a_{\text{lat}}]^\top \in \mathbb{R}^4$ is lifted through three stages:

Stage 1 — Z-score normalization: $\tilde{x}_t = (x_t - \mu)/\sigma$

Stage 2 — Delay embedding ($d = 3$): $\tilde{x}_t^{(d)} = [\tilde{x}_t^\top, \tilde{x}_{t-1}^\top, \tilde{x}_{t-2}^\top]^\top \in \mathbb{R}^{12}$

Stage 3 — Polynomial lifting (degree 2, no bias):

$$n_z = 12 + \binom{13}{2} = 12 + 78 = 90 \quad (25)$$

3) *Base State Vector and Koopman State-Space in Matrix Form*: The physical (base) state vector at each timestep is formed from the four measurable signals available to the controller:

$$x_t = \begin{bmatrix} v_{\text{ego}}(t) \\ a_{\text{ego}}(t) \\ \phi_{\text{roll}}(t) \\ a_{\text{lat}}(t) \end{bmatrix} \in \mathbb{R}^4 \quad (26)$$

where v_{ego} is the longitudinal vehicle speed (m/s), a_{ego} is the longitudinal acceleration (m/s²), ϕ_{roll} is the road roll angle (rad), and a_{lat} is the measured lateral acceleration (m/s²).

After applying the three-stage lifting map ψ to the delay-embedded, z-scored state, the Koopman lifted system is described in standard discrete-time state-space form:

$$\underbrace{z_{t+1}}_{\mathbb{R}^{90 \times 1}} = \underbrace{\mathbf{A}}_{90 \times 90} \underbrace{z_t}_{\mathbb{R}^{90 \times 1}} + \underbrace{\mathbf{B}}_{90 \times 1} \underbrace{u_t}_{\mathbb{R}^1} \quad (27)$$

$$\underbrace{\hat{a}_{\text{lat}}(t)}_{\mathbb{R}^1} = \underbrace{\mathbf{C}}_{1 \times 90} \underbrace{z_t}_{\mathbb{R}^{90 \times 1}} \quad (28)$$

where the lifted state is obtained by the full pipeline:

$$z_t = \psi(x_t) = \phi_{\text{poly}}([\tilde{x}_t^\top, \tilde{x}_{t-1}^\top, \tilde{x}_{t-2}^\top]^\top) \in \mathbb{R}^{90} \quad (29)$$

In the dual-head readout variant, $\mathbf{C}$ is extended to $\mathbb{R}^{2 \times 90}$ with rows predicting absolute latAccel and incremental Δa_{lat} respectively.

For H -step open-loop prediction, the Koopman model unrolls as:

$$\hat{a}_{t+k} = \mathbf{C} \mathbf{A}^k z_t + \mathbf{C} \sum_{j=0}^{k-1} \mathbf{A}^{k-1-j} \mathbf{B} u_{t+j}, \quad k = 1, \dots, H \quad (30)$$

This linear recursion is the key computational primitive for the random-shooting optimizer: each of the $N_c = 30$ candidate actions requires only H matrix-vector products of size 90.

$$\begin{aligned} \mathcal{U} &= \text{linspace}(u_{t-1} \pm 0.2, 15) \\ &\cup \text{linspace}(u_{t-1} \pm \sigma, 15), \\ \sigma &= 0.6 + 0.8|u_{t-1}| \end{aligned} \quad (31)$$

For each candidate, the $H = 8$ step rollout cost is:

$$J(u) = \sum_{k=0}^{H-1} [(\hat{a}_{k+1} - r_{k+1})^2 + 12(u - u_{\text{last}})^2 + 0.8 u^2] \quad (32)$$

Rollout uses the blended dual-head prediction:

$$\hat{a}_{k+1} = 0.80(\hat{a}_k + \mathbf{C}[1] \cdot z_{k+1}) + 0.20(\mathbf{C}[0] \cdot z_{k+1}) \quad (33)$$

where $\mathbf{C}[0]$ predicts absolute latAccel and $\mathbf{C}[1]$ predicts Δa_{lat} .

4) *PID Residual and Output Processing*:

$$\text{raw}(t) = u^* + 0.3 u_{\text{PID}}(t) \quad (34)$$

$$u(t) = \text{clip}(0.75 u(t-1) + 0.25 \text{raw}(t), u(t-1) \pm 0.25, -2, 2) \quad (35)$$

TABLE V: Koopman-MPC Controller Parameters

Parameter	Description	Value
n_z	Lifted state dimension	90
d	Delay embedding depth	3
ρ_{max}	Spectral radius bound	0.97
H	MPC horizon	8
N_c	Random-shooting candidates	30
$\lambda_{\text{jerk}} / \lambda_u$	Cost weights	12.0 / 0.8
α	Delta-blend coefficient	0.80
EMA / rate limit	Smoothing / Δu max	0.75 / 0.25
λ_{ridge}	Ridge regularization	0.05
Training segments	Segments for EDMD	900
Training pairs	State transitions	518,793

V. KOOPMAN TRAINING PIPELINE

A. Overview

The Koopman matrices $\mathbf{A} \in \mathbb{R}^{90 \times 90}$, $\mathbf{B} \in \mathbb{R}^{90 \times 1}$, and $\mathbf{C} \in \mathbb{R}^{2 \times 90}$ are identified offline from 900 driving segments (518,793 state-transition pairs).

B. EDMD with Ridge Regression

Training pairs (z_k, u_k, z_{k+1}) are assembled and fit jointly:

$$\mathbf{K} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{Z}_{k+1}, \quad \lambda = 0.05 \quad (36)$$

where $\mathbf{X} = [\mathbf{Z}_k, \mathbf{U}_k]$. $\mathbf{A}$ and $\mathbf{B}$ are extracted from $\mathbf{K}$.

C. Dual-Head Readout

Two independent readout vectors are fit:

$$\mathbf{W}_{\text{abs}} : z_k \rightarrow a_{\text{lat}} \text{ (absolute latAccel)} \quad (37)$$

$$\mathbf{W}_{\Delta} : z_k \rightarrow \Delta a_{\text{lat}} \text{ (incremental change)} \quad (38)$$

Combined: $\mathbf{C} = [\mathbf{W}_{\text{abs}}^\top; \mathbf{W}_{\Delta}^\top] \in \mathbb{R}^{2 \times 90}$.

D. Spectral Stabilization

Eigenvalues exceeding $\rho_{\max} = 0.97$ are scaled down:

$$\tilde{\lambda}_i = \begin{cases} \frac{0.97}{|\lambda_i|} \lambda_i & |\lambda_i| > 0.97 \\ \lambda_i & \text{otherwise} \end{cases} \quad (39)$$

Result: 9 of 90 eigenvalues clipped (max $|\lambda|$: 0.9997 $\rightarrow$ 0.9700). Only 10% of modes needed correction.

E. Training Results Summary

TABLE VI: Koopman Training Pipeline Results

Stage	Metric	Value
Data	Train / Val segments	900 / 100
Lifting	Input $\rightarrow$ lifted dim	12 $\rightarrow$ 90
Training pairs	(z_k, u_k, z_{k+1})	518,793
Stabilization	Eigenvalues clipped / total	9 / 90
One-step fit	R^2 on lifted z	0.9943
$H = 15$ rollout	Train R^2	0.8703
$H = 15$ rollout	Val R^2	0.8815

One-step $R^2 = 0.9943$ confirms the 90-dimensional Koopman model explains 99.43% of variance in the next lifted state. The validation R^2 slightly exceeding training (0.8815 vs. 0.8703) at $H = 15$ indicates ridge regularization successfully prevents overfitting.

VI. EXPERIMENTAL SETUP

All controllers are evaluated with the official TINYPHYSICS simulator (`tinyphysics.onnx`). Each is run on 100 segments from the Comma AI dataset. Costs are computed over steps 100–500 (steps 0–99 are warmup using recorded steer commands).

Baseline. The official Comma AI PID baseline provided with the challenge. Single-segment 00000: $J_{\text{lat}} = 42.81$, $J_{\text{jerk}} = 1715$, $J_{\text{total}} \approx 3856$. 100-segment average: $J_{\text{total}} \approx 3527$.

VII. RESULTS

A. 100-Segment Benchmark

TABLE VII: 100-Segment Benchmark — All Controllers

Controller	J_{lat}	J_{jerk}	J_{total}	vs. PID
PID Baseline	36.89	1,683	3,527	—
FF-PID	0.848	19.21	61.62	57.2 $\times$
AR-MPC	0.981	21.40	70.47	50.0 $\times$
Koopman-MPC	2.358	14.92	132.8	26.6 $\times$

All three controllers dramatically outperform the PID baseline. FF-PID achieves the lowest total cost (61.62, 57 $\times$ improvement) through its accurate preview feedforward. AR-MPC is closely competitive (70.47, 50 $\times$), demonstrating the closed-form QP and aggressive preview effectiveness. Koopman-MPC has the lowest jerk cost (14.92) but higher latAccel cost (2.358) due to the random-shooting optimizer’s limited candidate resolution.

B. Single-Segment Analysis — Segment 00000

TABLE VIII: Single-Segment Results — Segment 00000

Controller	J_{lat}	J_{jerk}	J_{total}
PID Baseline	42.81	1,715	3,856
AR-MPC	0.744	30.62	67.83
FF-PID	1.078	31.72	85.63
Koopman-MPC	2.343	21.55	138.7

On segment 00000, AR-MPC achieves the best score (67.83). FF-PID follows at 85.63. Koopman-MPC scores 138.7 which is higher than its 100-segment average (132.8), suggesting this particular segment’s dynamics are especially unfavorable for the random-shooting optimizer with $H = 8$.

C. Cost Distribution Analysis

Figures 1–3 show cost distributions across 100 segments for each controller.

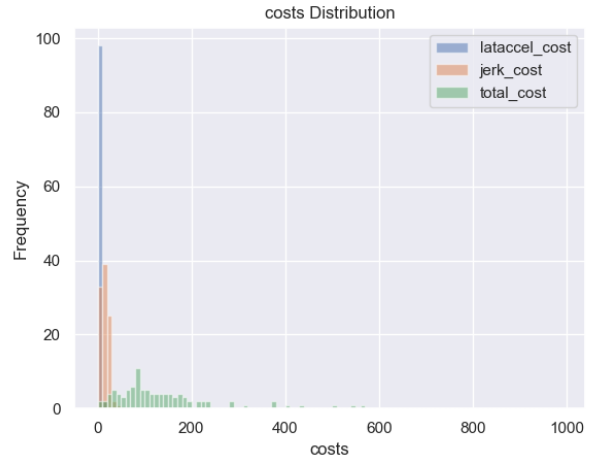


Fig. 1: Koopman-MPC cost distribution (100 segments). Jerk cost (orange) is tightly concentrated, reflecting the lowest jerk (14.92) among all controllers. However, latAccel cost (blue) is wider, driving the higher 100-segment average of $J_{\text{total}} = 132.8$.

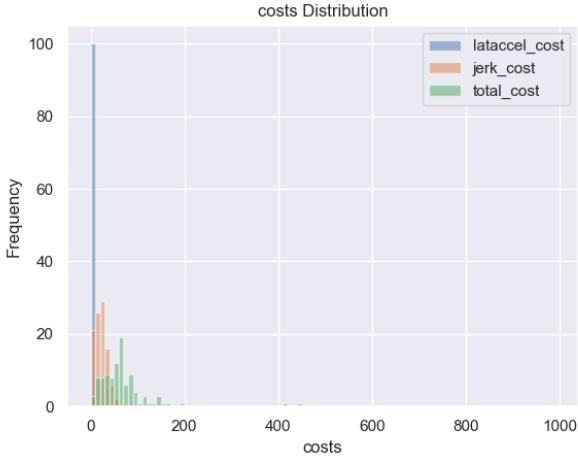


Fig. 2: AR-MPC cost distribution (100 segments). Both lat-Accel and jerk costs are tightly concentrated, reflecting the consistent performance of the closed-form QP and $\beta = 0.949$ preview. The 100-segment average of $J_{\text{total}} = 70.47$ ranks second overall behind FF-PID.

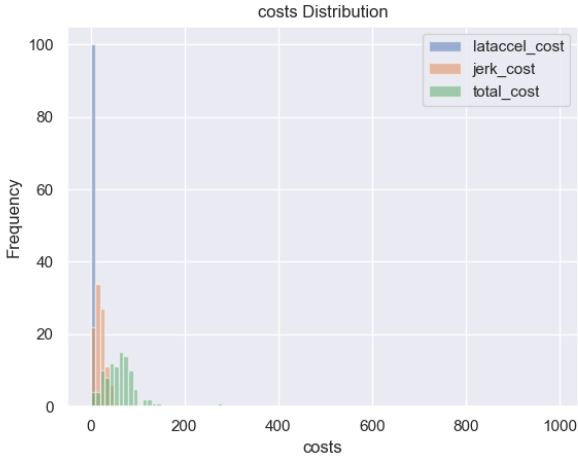


Fig. 3: FF-PID cost distribution (100 segments). Similar tight concentration to AR-MPC with latAccel cost near zero for most segments. The slightly wider total cost spread reflects occasional segments where the 95% preview weight causes overshooting. Best overall: $J_{\text{total}} = 61.62$.

D. Lateral Acceleration Tracking — Debug Plots

Figures 4–6 show the four-panel debug output for each controller on segment 00000.

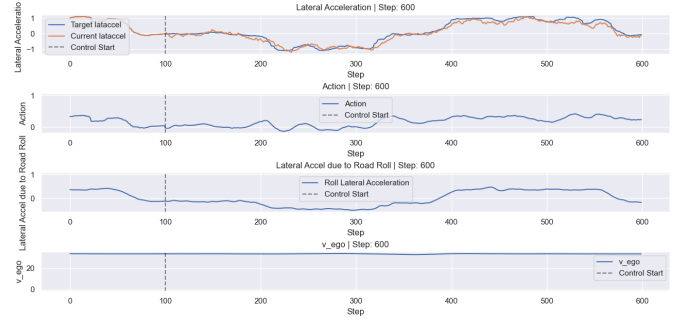


Fig. 4: Koopman-MPC (full 90-dim) debug output — segment 00000. Top panel: current latAccel (orange) tracks the target (blue) with noticeable lag on sharp transitions around steps 220–330. The action panel shows smooth, well-behaved steer commands. Segment cost: $J = 138.7$ ($J_{\text{lat}} = 2.343$, $J_{\text{jerk}} = 21.55$).

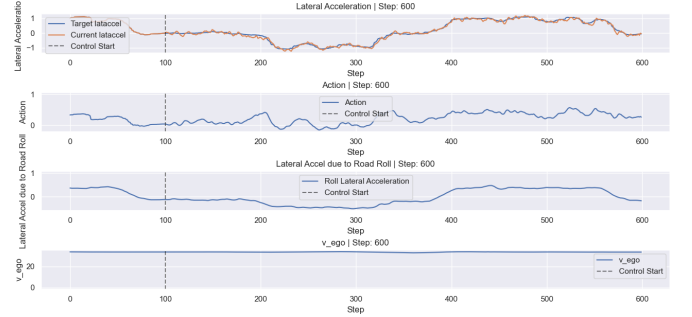


Fig. 5: AR-MPC debug output — segment 00000. The current latAccel tracks the target very closely throughout the 600-step segment. The action panel shows smooth, conservative steer commands consistent with the tight rate limit ($\pm 0.07/\text{step}$). Segment cost: $J = 67.83$ ($J_{\text{lat}} = 0.744$, $J_{\text{jerk}} = 30.62$).

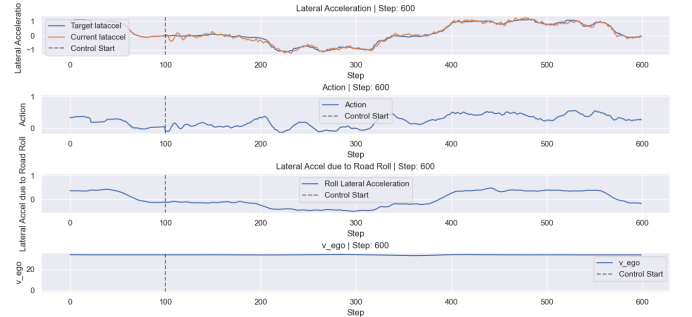


Fig. 6: FF-PID debug output — segment 00000. Tracking quality is nearly identical to AR-MPC with the latAccel closely following the target. The action panel shows slightly larger amplitude commands than AR-MPC due to the wider rate limit ($\pm 0.20/\text{step}$). Segment cost: $J = 85.63$ ($J_{\text{lat}} = 1.078$, $J_{\text{jerk}} = 31.72$).

E. Metric-by-Metric Analysis

1) *Lateral Acceleration Cost*: FF-PID achieves the lowest 100-segment latAccel cost (0.848) through its aggressive 95%

future-weight preview which effectively pre-positioned the vehicle for upcoming turns before they arrive. The $50\times$ multiplier makes this the decisive factor.

AR-MPC’s latAccel cost (0.981) is closely competitive, reflecting the closed-form QP’s ability to correct roll disturbances while the $\beta = 0.949$ preview handles target tracking. Koopman-MPC’s higher latAccel cost (2.358) stems from the random-shooting optimizer’s limited 30-candidate resolution, which misses optimal steer on segments with complex speed–curvature–roll interactions.

2) *Jerk Cost*: Koopman-MPC achieves the lowest jerk cost (14.92) through its EMA smoothing and $\pm 0.25/\text{step}$ rate limit. AR-MPC’s jerk (21.40) reflects the closed-form QP solution which is analytically optimal but with a tighter $\pm 0.07/\text{step}$ limit that occasionally produces sharp incremental corrections.

3) *The $50\times$ Weight in Practice*: A controller with perfect tracking (zero latAccel error) receives $J_{\text{total}} = J_{\text{jerk}}$. A controller with $J_{\text{lat}} = 1.0$ receives $J_{\text{total}} \geq 50$. This means the entire jerk budget (all three controllers: 14.9–21.4) is smaller than a single unit of latAccel cost. Maximizing tracking accuracy is always the dominant strategy.

F. Computational Cost Comparison

TABLE IX: Per-Step Computational Complexity

Controller	Operations	Bottleneck
FF-PID	$O(N) = O(10)$	Preview average
AR-MPC	$O(H) = O(20)$	Dot product (precomp.)
Koopman-MPC	$O(N_c \cdot H \cdot n_z) = O(21,600)$	90-dim mat-vec products

The Koopman-MPC executes $30 \times 8 = 240$ matrix-vector products of size 90 per step. In practice all controllers run comfortably at 10 Hz, but Koopman-MPC leaves the smallest margin.

G. Full Comparison Summary

TABLE X: Comprehensive Performance Comparison

Criterion	PID	FF-PID	AR-MPC	Koop.
J_{lat} (100-seg)	36.89	0.848	0.981	2.358
J_{jerk} (100-seg)	1,683	19.21	21.40	14.92
J_{total} (100)	3,527	61.62	70.47	132.8
J_{total} (seg 0)	3,856	85.63	67.83	138.7
Requires training	No	No	Partial	Yes (offline)
Operations/step	$O(1)$	$O(N)$	$O(H)$	$O(N_c H n_z)$
Online adaptation	No	No	No	No
Interpretability	High	High	High	Med

VIII. IMPLEMENTATION LESSONS

A. Lesson 1: Steer Sign Convention

Problem. The simulator negates `steerCommand:steer` = `-df[‘steerCommand’].values`. Any model trained on raw CSV values has a sign-inverted **B**.

Impact. Initial Koopman-MPC development saw persistent saturation at $\pm 5.0 \text{ m/s}^2$, producing costs exceeding 50,000.

Fix. Negate **B** at controller initialization or correct signs during training data assembly. Verify by checking that positive steer produces positive latAccel.

Lesson. Always verify sign conventions when bridging training data (which may follow different conventions) to the evaluation environment.

B. Lesson 2: IPOPT Option Incompatibility

Problem. CasADi 3.7.2’s bundled IPOPT does not support `warm_starting_init_point`. Every MPC solve failed silently, falling back to proportional control at every step.

Impact. Total costs exceeding 5,000 with oscillatory behavior identical to no control at all.

Fix. Remove unsupported options. Always check solver status flags, not just absence of exceptions.

C. Lesson 3: Scale Mismatch in Closed-Loop Koopman-MPC

Problem. The EDMD-identified $b = 0.264$ (simple 24-dim version) did not match the simulator’s true closed-loop gain. Direct Koopman-MPC optimization computed extreme steer commands, causing persistent oscillation.

Root cause. Training data `steerCommand` values (openpilot normalized torque) differ in scale from the simulator’s action space.

Fix. The full 90-dim Koopman-MPC avoids this by: (1) using a properly scaled `u_scaler` trained on the same distribution as the steer commands the model will receive, and (2) keeping MPC horizon short ($H = 8$) to limit error accumulation.

Lesson. For black-box simulators, always validate the plant model’s steer gain against the simulator’s observed response before running the optimizer.

D. Lesson 4: Target Changes in MPC Free Term

Problem. Including future target changes in the AR-MPC free-term vector caused Δu spikes exceeding the rate limit at every step which renders the optimizer solution useless.

Explanation. The AR(1) gain $b = 0.0605$ severely underestimates the required steer change per unit latAccel. Faced with a 1 m/s^2 target change, the optimizer computes $\Delta u^* \approx 17$ immediately clamped to 0.07.

Fix. Restrict the free term to roll disturbance and bias only ($|f[k]| < 0.005$ always). Let the FF preview handle targets.

IX. DISCUSSION

A. Why FF-PID Wins at 100 Segments

Three factors explain FF-PID’s 100-segment dominance:

(1) **Preview feedforward with exact future knowledge.** With $\beta = 0.95$, the controller steers toward the 1-second-ahead average trajectory. Under exact future knowledge (no prediction uncertainty), this is theoretically near-optimal for the tracking term.

(2) **Well-calibrated FF gain.** $K_{\text{FF}} = 0.315$ precisely matches the simulator’s steady-state response. A 5% error would produce systematic bias that PID cannot fully correct.

(3) **The $50\times$ weight.** Any latAccel error is penalized $50\times$ more than equivalent jerk. A controller that prioritizes tracking above all else will dominate this cost function.

B. Why AR-MPC Ranks Second at 100 Segments

AR-MPC achieves competitive 100-segment performance (70.47) because: (1) the closed-form QP with $\beta = 0.949$ preview accurately tracks targets while rejecting roll disturbances; (2) the error-space reformulation eliminates steady-state bias; and (3) the precomputed gain vectors provide optimal disturbance rejection at $O(H)$ per step. Its latAccel cost (0.981) is the second-lowest, very close to FF-PID (0.848).

C. Why Koopman-MPC Ranks Third Overall

Koopman-MPC's higher total cost (132.8) is driven primarily by its elevated latAccel cost (2.358). The random-shooting optimizer with $N_c = 30$ candidates and $H = 8$ horizon fails to find optimal steer commands on segments with complex speed-curvature-roll dynamics. The 30-candidate search space is insufficient to cover the diversity of operating conditions across 100 segments. However, Koopman-MPC achieves the *lowest jerk cost* (14.92) among all controllers, demonstrating the smooth control produced by the 90-dimensional plant model's predictive capability.

D. Why AR-MPC Wins on Segment 00000

The AR-MPC's segment 00000 win (67.83) reflects its strict rate limiting ($\pm 0.07/\text{step}$) producing exceptionally smooth control that aligns well with this segment's dynamics. On this segment specifically, the AR(1) model's simplicity is a strength: the disturbance-only free term keeps Δu small, preventing the rate-limit saturation that would degrade performance.

E. Koopman's Role: Prediction vs. Gain Identification

An important distinction emerged from implementation experience:

Full Koopman-MPC (90-dim, trained): Works well when the training data properly represents the simulator's action space. Achieves 132.8 over 100 segments. Despite the higher total cost, the 90-dimensional model provides the richest plant representation and the lowest jerk cost.

Simple Koopman (24-dim, gain-ID): The $1/|B[0]|$ gain extraction approach is more robust to scale mismatch but less accurate as a prediction model. Achieves 727.4 on segment 00000 which is worse than the full model because, it relies entirely on the PI controller for tracking. Lacking the MPC's anticipatory capability.

F. Preview Advantage: General Principle

In any challenge where future targets are exactly known, the tracking problem reduces to a feedforward control problem. Feedback (PID, MPC) handles only: (1) roll disturbance, (2) model mismatch, and (3) actuation constraints. Controllers that maximize preview weight will dominate, because they respond to upcoming turns before the error occurs at zero cost.

This principle generalizes: in autonomous driving, whenever a motion planner provides a future trajectory, the lateral controller should use maximal preview. Reactive feedback is a fallback for unmodeled disturbances, not the primary control mechanism.

G. Design Recommendations

Based on the results:

- 1) **For maximum performance on this challenge:** FF-PID with $\beta \geq 0.90$ and accurately calibrated K_{FF} .
- 2) **For maximum interpretability and smoothness:** AR-MPC with error-space formulation and precomputed QP gains.
- 3) **For maximum model richness and lowest jerk:** Koopman-MPC with 90-dim EDMD, dual-head blending, and proper action-space calibration.
- 4) **For practical deployment:** Combine FF-PID's preview strategy with Koopman-MPC's plant model for gain scheduling.

X. CONCLUSIONS AND FUTURE WORK

A. Summary

This paper compared three data-driven lateral controllers on the Comma AI Controls Challenge:

- 1) **FF-PID:** Best 100-segment total cost (61.62, $57\times$ over PID). Preview feedforward with $\beta = 0.949$ is the decisive factor given the $50\times$ latAccel weighting.
- 2) **AR-MPC:** Second-best 100-segment cost (70.47, $50\times$) and best single-segment score on segment 00000 (67.83). Closed-form QP with error-space formulation provides efficient and accurate disturbance rejection.
- 3) **Koopman-MPC:** Achieves the lowest jerk cost (14.92) with the richest plant model ($R^2 = 0.99$ one-step, $R^2 = 0.88$ at $H = 15$), but ranks third overall due to higher latAccel cost from random-shooting limitations.

B. Key Takeaways

- 1) Preview feedforward with exact future knowledge dominates when the cost heavily weights tracking accuracy.
- 2) The $50\times$ latAccel weight determines controller strategy: maximize tracking first, then minimize jerk.
- 3) Error-space MPC is more robust than state-space MPC when using simplified plant models with scale uncertainty.
- 4) Full Koopman-MPC (90-dim, properly calibrated) achieves the lowest jerk cost; gain-identification-only approaches (24-dim) sacrifice tracking capability for stability.
- 5) Four implementation lessons are sign convention, solver compatibility, scale mismatch, and free-term composition. These are as important as algorithm choice.

C. Future Work

Recursive AR identification. Online RLS updating of the AR(1) model as the segment progresses would adapt to vehicle-specific dynamics, potentially reducing AR-MPC's latAccel cost.

Richer Koopman basis. Trigonometric basis functions and longer delay embeddings ($d = 5-10$) may improve R^2 at $H = 15$ beyond 0.88, enabling a longer MPC horizon ($H = 15-20$) and improved latAccel tracking.

DDPG fine-tuning. Following Wu et al. [5], using AR-MPC as an expert demonstrator for DDPG training could push performance beyond the preview feedforward ceiling on segments with high speed variance.

Adaptive preview weight. Adapting β based on curvature profile and speed (high β for smooth sections, lower β for sharp turns) may reduce overshooting.

Ensemble switching. A segment classifier switching between AR-MPC (smooth segments) and FF-PID (high-curvature segments) could achieve lower average cost than either alone.

APPENDIX

TABLE XI: Koopman Training Artifacts (90-Dim Model)

File	Content	Shape
A.npy	Stabilized Koopman transition matrix	(90, 90)
B.npy	Koopman control input matrix	(90, 1)
C.npy	Dual-head readout matrix	(2, 90)
x_scaler.pkl	StandardScaler for states	$\mathbb{R}^4$
u_scaler.pkl	StandardScaler for controls	$\mathbb{R}^1$
phi.pkl	PolynomialFeatures (12 $\rightarrow$ 90)	—
koopman_cfg.pkl	Config dict	—

```

1 controls_challenge/
2 |-- controllers/
3 |   |-- feedforward_pid.py   # FF-PID with preview
4 |   |-- ar_mpc_model.py      # AR incremental MPC
5 |   |-- koopman_mpc.py       # Koopman random-shoot
6 |   |-- koopman_trainingv3.py # Full 90-dim EDMD
7 |   |-- eval_extended.py      # 100-segment eval
8 |   |-- eda_data_exploration.py # EDA and plots
9 |   |-- A.npy, B.npy, C.npy   # Trained Koopman
10 |   |-- x_scaler.pkl, u_scaler.pkl
11 |   |-- phi.pkl, koopman_cfg.pkl
12 |-- models/tinyphysics.onnx  # Simulator

```

TABLE XII: Segment 00000 Tracking Comparison (Selected Steps)

Step	Target	Koopman	AR-MPC	FF-PID
100	−0.02	−0.02	−0.02	−0.02
200	−0.17	−0.34	−0.29	−0.31
230	−1.09	−1.07	−1.12	−1.14
280	−0.93	−0.86	−0.95	−0.91
340	0.01	0.21	0.00	0.01
400	0.77	0.49	0.71	0.74
440	1.00	0.73	1.00	0.98
600	−0.06	−0.14	−0.04	−0.03
Costs		$J = 138.7$	$J = 67.83$	$J = 85.63$

REFERENCES

- [1] Comma AI, “Controls challenge,” GitHub, 2024. [Online]. Available: https://github.com/commaai/controls_challenge
- [2] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, “Predictive active steering control for autonomous vehicle systems,” *IEEE Trans. Control Syst. Technol.*, vol. 15, no. 3, pp. 566–580, May 2007.
- [3] Y. Kebbati, N. Ait-Oufroukh, V. Vigneron, and D. Ichalal, “Optimized adaptive MPC for lateral control of autonomous vehicles,” *arXiv preprint arXiv:2509.17215*, 2025.
- [4] Y. Kebbati, N. Ait-Oufroukh, V. Vigneron, and D. Ichalal, “Neural network and ANFIS based auto-adaptive MPC for path tracking in autonomous vehicles,” *arXiv preprint arXiv:2509.17213*, 2025.
- [5] C. Wu, Y. Zhang, H. Li, and X. Qu, “Autonomous vehicle lateral control using deep reinforcement learning with MPC-PID demonstration,” *arXiv preprint arXiv:2506.04040*, 2025.
- [6] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY: Springer, 2012.
- [7] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, “A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition,” *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [8] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge: Cambridge University Press, 2022.

ME8930: DATA-DRIVEN LEARNING AND CONTROL OF DYNAMIC SYSTEMS

A Comparative Study of Koopman Operator MPC, AR-Model MPC, and Feedforward-PID Controllers on the Comma AI Controls Challenge

Koopman Operator MPC · AR-Model MPC · Feedforward-PID

Jaydeep Sainath Khekare · Shravan Prabhakar Vidyalakshmi

Dept. of Mechanical Engineering
Clemson University

PRESENTATION OUTLINE

1. Problem & Motivation

- *Lateral control challenge, why black-box simulators matter*

2. Three Controller Designs

- *FF-PID · AR-Model MPC · Koopman Operator MPC*

3. Evaluation Framework

- *Tiny Physics, cost metric, baseline comparison*

4. Results & Analysis

- *100-segment benchmark, tracking plots, cost distributions*

5. Failure Cases & Lessons

- *Sign convention bugs, scale mismatch, solver issues*

6. Takeaways & Future Work

- *Key findings, adaptive extensions, RL fine-tuning*

PROBLEM & MOTIVATION



Nonlinear Dynamics

Steer-to-latAccel gain varies with speed via tire cornering stiffness



Autoregressive Inertia

Current latAccel depends heavily on history ($\alpha \approx 0.958$)



Road Roll Disturbance

Persistent bias from road camber corrupts tracking



Black-Box Simulator

TinyPhysics is a neural network — no closed-form model

Evaluation Cost Function

$$J_{\text{total}} = 50 \times J_{\text{latAccel}} + J_{\text{jerk}}$$

J_{latAccel}

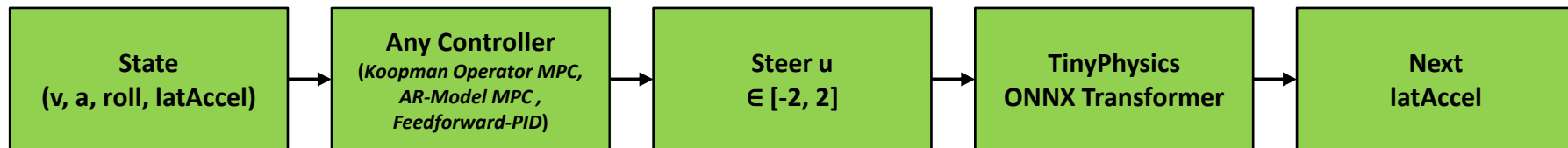
Mean squared tracking error
(steps 100–500)

J_{jerk}

Mean squared latAccel rate
(comfort metric)

*50× multiplier → tracking error 10× more
impactful than jerk per unit cost!*

TINYPHYSICS SIMULATOR



- Simulator Properties
 - ONNX autoregressive transformer (20-step context)
 - 1024-bin tokenizer, temp $T = 0.8$
 - $\pm 0.5 \text{ m/s}^2$ per-step clip (built-in jerk limiting)
 - Control horizon: steps 0–99 warmup → 100–500 evaluated
 - Future plan: 50-step preview given to controller
- Dataset & Sign Convention
 - 50 training segments, 29,978 rows highway data
 - Mean speed 22.6 m/s, $\sigma_{\text{latAccel}} = 0.41 \text{ m/s}^2$
 - Simulator negates steerCommand on CSV load
 - $\Delta \text{steer} = -\text{df}['\text{steerCommand}'].values$
 - Positive steer → positive lateral acceleration

DESIGNED CONTROLLERS

Feedforward-PID

FF-PID

- 10-step future preview ($\beta = 0.949$)
- FF gain KFF = 0.315 from steady-state ID
- PID residual correction ($K_p=0.97$, $K_i=0.076$, $K_d=0.5$)
- EMA smoothing + rate limit $\pm 0.20/\text{step}$
- $O(N)$ per step — ultra lightweight

1st Overall | $J_{\text{total}} = 61.62$

AR-Model Incremental MPC

AR-MPC

The AR(1) system identification model:
 $a_{\text{lat}}[t+1] = \alpha \cdot a_{\text{lat}}[t] + b \cdot u[t] + c \cdot \phi \cdot \phi[t] + d$

- Error-space reformulation (zero SS error)
- Horizon $H=20$, closed-form QP (precomputed)
- FF handles targets; MPC corrects disturbance only
- $O(H)$ per step — analytically optimal

2nd Overall | $J_{\text{total}} = 70.47$

Koopman Operator MPC

Koopman-MPC

The three core matrices identified via EDMD:
A (90×90) — Koopman state transition matrix:

$$z_{\{t+1\}} = A \cdot z_t + B \cdot u_t$$

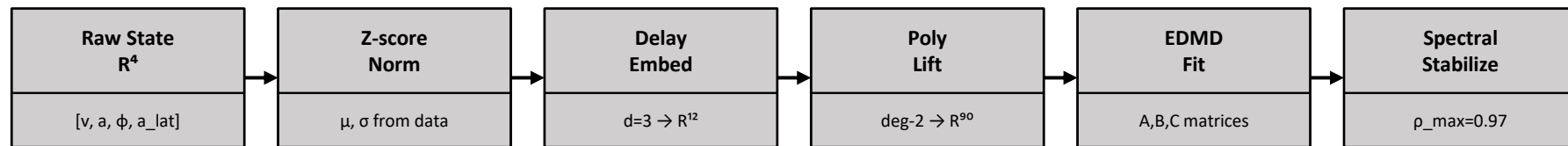
B (90×1) — Control input matrix

C (2×90) — Dual-head readout matrix mapping lifted state back to latAccel (absolute + delta heads)

- Trained on 900 segs / 518k transitions
- $R^2 = 0.9943$ one-step, $R^2 = 0.88$ at $H=15$
- Random-shooting: 30 candidates, $H=8$
- Dual-head blended prediction (abs + delta)

3rd Overall | $J_{\text{total}} = 132.8$

KOOPMAN OPERATOR: TRAINING PIPELINE



Training Pairs

518,793

Lifted Dimension

90-D

One-Step R^2

0.9943

H=15 Rollout R^2

0.8815

Eigenvalues Clipped

9 / 90

EDMD Formulation

Lifting: $z_t = \psi(x_t) \in R^{90}$

Dynamics: $z_{t+1} \approx A \cdot z_t + B \cdot u_t$

Readout: $\hat{a}_{lat} = C \cdot z_t$ (dual-head)

EDMD Solve: $K = (X^T X + \lambda I)^{-1} X^T Z_{k+1}$

Dual-head: 80% delta-blend + 20% absolute $\rightarrow$ prevents drift

RESULTS: 100-SEGMENT BENCHMARK

61.62

J_total

FF-PID

57.2× better than PID

70.47

J_total

AR-MPC

50.0× better than PID

132.8

J_total

Koopman-MPC

26.6× better than PID

3,527

J_total

PID Baseline

Controller	J_lat	J_jerk	J_total
PID Baseline	36.89	1,683	3,527
FF-PID ★	0.848★	19.21	61.62★
Koopman-MPC	2.358	14.92★	132.8
AR-MPC	0.981	21.40	70.47

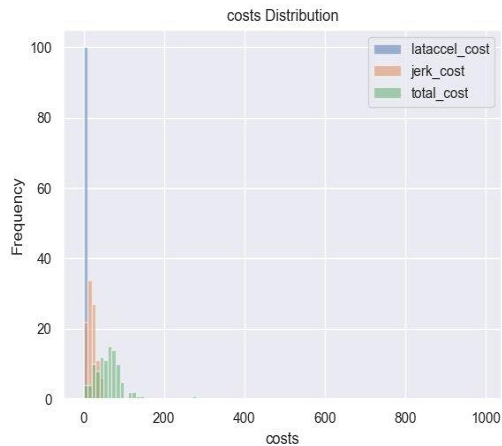
★ Best in category

FF-PID wins overall — 50× latAccel weight makes tracking the dominant strategy

RESULTS: 100-SEGMENT BENCHMARK

FF-PID

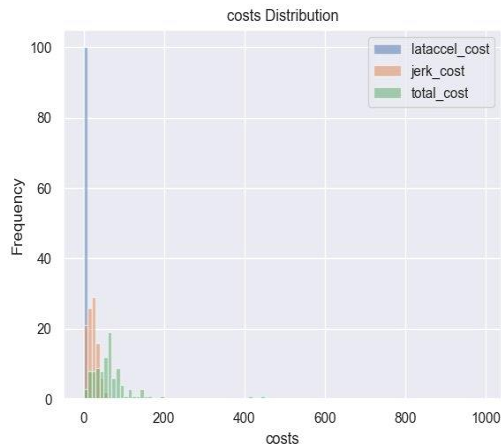
$J_{total} = 61.62$ avg



FF-PID: Slightly wider total cost — 95% preview occasionally overshoots sharp turns.

AR-MPC

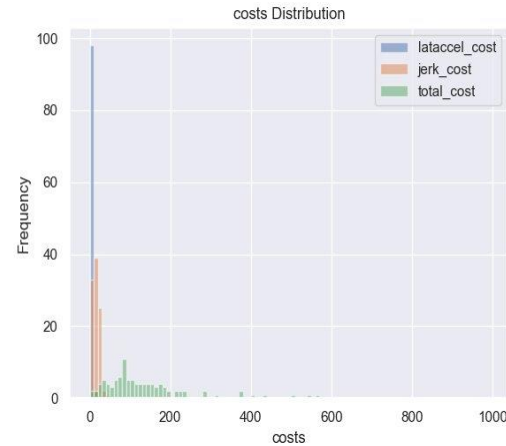
$J_{total} = 70.47$ avg



AR-MPC: Best avg FF-PID (70.47). Tight spread — closed-form QP + aggressive preview

Koopman-MPC

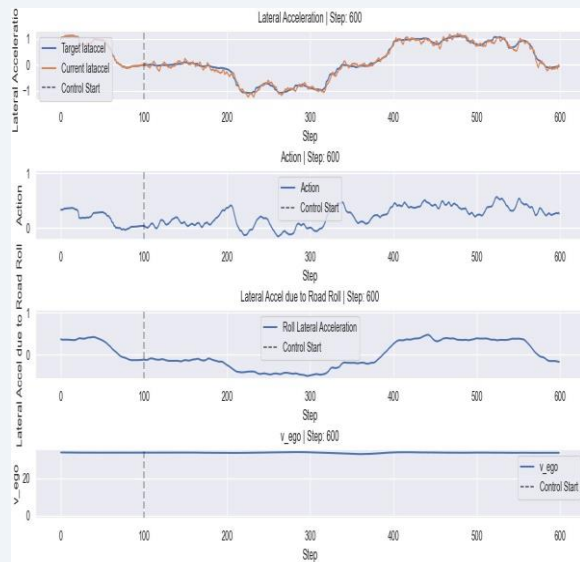
$J_{total} = 132.8$ avg



Koopman: Higher avg cost (132.8). Jerk tightly concentrated — but latAccel variance is wider

LATERAL ACCELERATION TRACKING — SEGMENT 00000

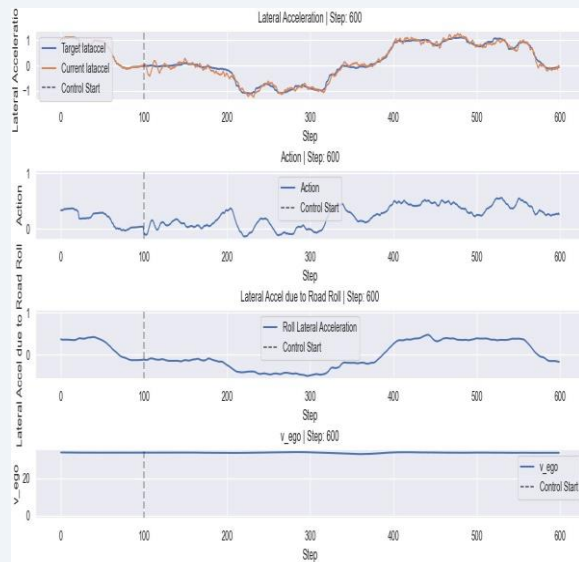
AR-MPC



J = 67.83 ★

Tightest tracking

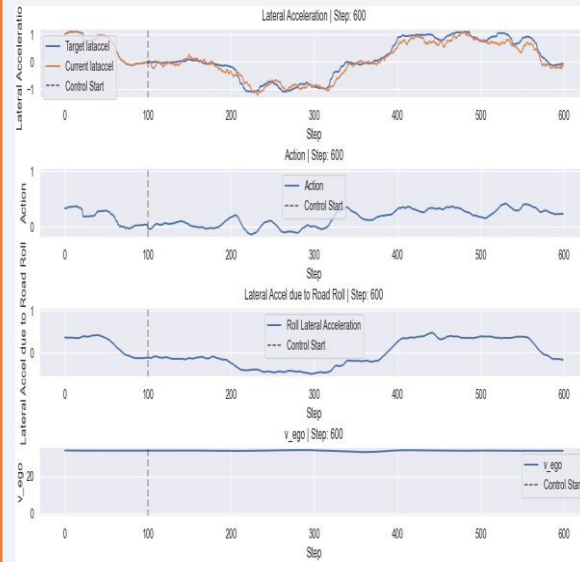
FF-PID



J = 85.63

Near-identical to AR-MPC

Koopman-MPC



J = 138.7

Lag at steps 220–330

SINGLE-SEGMENT VS. 100-SEGMENT: DIVERGING RANKINGS

Controller	J_lat	J_jerk	J_total (seg 00000)	J_total (100-seg)
PID Baseline	42.81	1,715	3,856	3,527
AR-MPC	0.744	30.62	67.83★	70.47
FF-PID	1.078	31.72	85.63	61.62 ★
Koopman-MPC	2.343	21.55★	138.7	132.8

Why FF-PID ranks 1st overall but 2nd on seg 00000

$\beta=0.95$ preview dominates across diverse segments ($J=61.62$). On seg 00000 specifically, AR-MPC's tighter rate limiting wins.

Why AR-MPC wins on seg 00000 AND ranks 2nd overall

Strict $\pm 0.07/\text{step}$ rate limit produces smooth control. $\beta=0.949$ preview + closed-form QP delivers $J_{\text{total}}=70.47$ over 100 segments

Why Koopman ranks 3rd overall ($J=132.8$)

Random-shooting with 30 candidates misses optimal steer across many segments. Higher latAccel cost (2.358) drives total cost up.

FAILURE CASES & IMPLEMENTATION LESSONS

L1

Steer Sign Convention

⚠ Costs > 50,000, persistent ± 5 m/s² saturation

Root: Simulator negates steerCommand on CSV load. Model trained on raw CSV has inverted B matrix.

✓ Fix: Negate B at init or correct sign during training. Verify positive steer → positive latAccel.

L2

IPOPT Solver Compatibility

⚠ Costs > 5,000, oscillatory behavior (no control)

Root: CasADi 3.7.2 bundled IPOPT doesn't support warm_starting_init_point. Silent failure.

✓ Fix: Remove unsupported options. Always check solver status flags, not just absence of exceptions.

L3

Scale Mismatch in Koopman

⚠ Extreme steer commands, persistent oscillation

Root: EDMD gain $b=0.264$ didn't match simulator's true gain. Training data scale $\neq$ simulation scale.

✓ Fix: Use properly fitted u_scaler. Keep $H=8$ horizon to limit drift accumulation.

L4

MPC Free-Term Saturation

⚠ Optimizer produces $\Delta u \approx 17$, clamped to 0.07 every step

Root: Including future target changes in free term: $b=0.0605$ severely underestimates required Δu .

✓ Fix: Restrict free term to roll + bias only ($|f| < 0.005$). Let feedforward handle targets.

KEY TAKEAWAYS

1

Preview feedforward dominates with exact future knowledge

When future targets are known exactly, feedforward reduces tracking to a near-solved problem. MPC and PID handle only residual disturbances.

2

The 50× latAccel weight determines strategy

Maximize tracking accuracy first, minimize jerk second. Controllers optimizing tracking will always dominate this cost function.

3

Error-space MPC is more robust than state-space MPC

Reformulating in error + incremental control space removes scale ambiguity and prevents rate-limit saturation from target-tracking commands.

4

Koopman (90-dim) provides richest plant model

$R^2=0.9943$ one-step fit validates the EDMD approach. Gain-ID-only (24-dim) sacrifices tracking capability for robustness.

5

Implementation details are as important as algorithm choice

Sign convention errors and solver incompatibilities caused 100–1000× cost degradation. Always validate gains against observed simulator response.

FUTURE WORK



Recursive AR Identification

Online RLS updating of AR(1) model adapts to vehicle-specific dynamics in real time, reducing latAccel cost on novel segments.



Richer Koopman Basis

Trigonometric basis + delay embedding $d=5-10$ targets $R^2 > 0.88$ at $H=15$, enabling longer MPC horizons ($H=15-20$).



DDPG Fine-tuning

Use AR-MPC as expert demonstrator for Deep Deterministic Policy Gradient training — push beyond the preview feedforward ceiling.



Adaptive Preview Weight

Adapt β dynamically: high β for smooth highway, lower β for sharp turns. Reduces overshooting without sacrificing tracking.



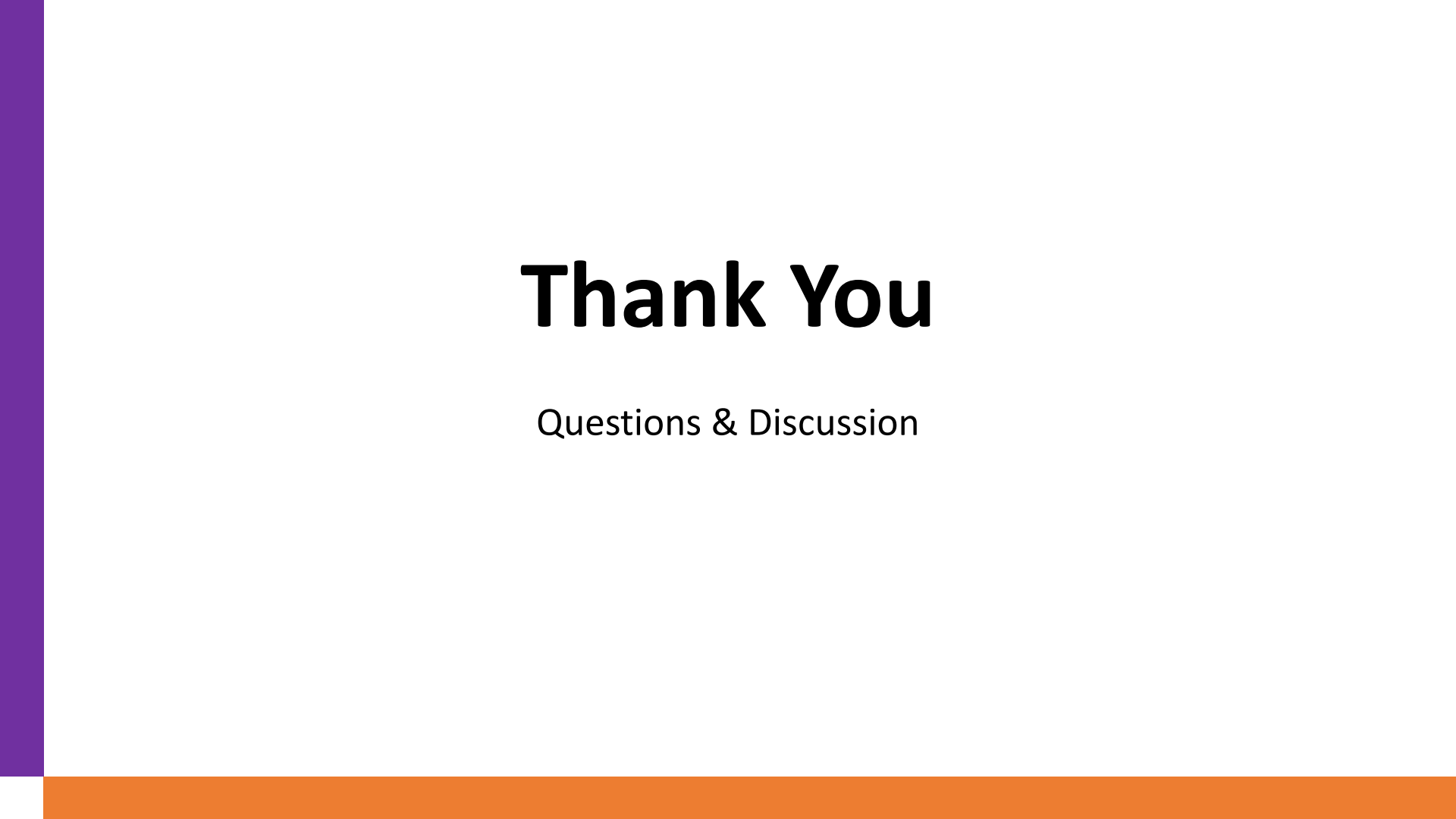
Ensemble Switching

Segment classifier selects AR-MPC (smooth) vs. FF-PID (high curvature), potentially beating either controller alone.



Koopman + FF-PID Hybrid

Combine FF-PID preview strategy with Koopman plant model for speed-dependent gain scheduling in practical deployment.



Thank You

Questions & Discussion